

◆ MODULE 1

1. Epipolar Geometry and RANSAC

Explanation of Epipolar Geometry & Uses:

- **Definition:** Epipolar geometry is the intrinsic projective geometry between two camera views. It describes the geometric relationship that exists entirely independent of the scene structure, relying only on the cameras' internal parameters and relative pose.
- **The Epipolar Plane:** Defined by the 3D scene point (X) and the two camera optical centers (C_1, C_2). This plane intersects the image planes to form epipolar lines.
- **The Epipole:** The point on the image plane where the baseline (line connecting the two camera centers) intersects. It effectively represents the image of one camera center as seen by the other camera.
- **Epipolar Constraint:** The fundamental rule stating that for a given point x in the first image, its corresponding matching point x' in the second image *must* lie on the corresponding epipolar line.
- **Fundamental Matrix (F):** The algebraic representation of this geometry. The constraint is written as $x'^T F x = 0$.
- **Use in Robotic Vision (Stereo Matching):** It reduces the correspondence search problem from a 2D search (entire image) to a 1D search (along the epipolar line), drastically reducing computational cost and false matches.
- **Use in 3D Reconstruction:** It is the prerequisite for triangulation. Once the geometry is known, matches can be converted into 3D depth.

RANSAC for Robust Feature Matching:

- **The Problem:** In stereo vision, simple feature matching (e.g., matching SIFT descriptors) produces many outliers due to repetitive textures or noise.
- **Random Sampling:** RANSAC (Random Sample Consensus) starts by selecting the *minimum* number of random point correspondences required to solve the geometric model (e.g., 7 or 8 points for the Fundamental Matrix).
- **Model Hypothesis:** It computes a candidate Fundamental Matrix from this random minimal subset.
- **Verification (Inlier Counting):** It tests all other putative matches in the dataset against this candidate matrix using the epipolar constraint ($x'^T F x \approx 0$). Matches that satisfy the constraint are counted as "inliers."
- **Iterative Refinement:** This process repeats for N iterations. The model with the highest number of inliers is selected as the winner.
- **Final Estimation:** The final Fundamental Matrix is re-computed using *all* the inliers (via Least Squares) to minimize error and maximize robustness.

Reference: Szeliski pp. 703–708 (Epipolar), 510–512 (RANSAC).

2. DL-based Segmentation in Unstructured Environments

Deep Learning Segmentation Models:

- **Semantic Segmentation:** Assigns a class label (e.g., "rock," "sand," "grass") to every pixel. It provides a dense classification map essential for drivability analysis.

- **Fully Convolutional Networks (FCN):** Adapts classification networks (like VGG) by replacing dense layers with convolutional ones, allowing the network to output a spatial heatmap rather than a single label.
- **U-Net (Encoder-Decoder):** The standard for robotic vision.
 - **Encoder:** Downsamples the image to capture "what" is present (context).
 - **Decoder:** Upsamples the features to capture "where" it is (localization).
 - **Skip Connections:** Critical links that copy high-resolution features from the encoder directly to the decoder to preserve edge details lost during pooling.
- **Instance Segmentation (Mask R-CNN):** Goes beyond semantic segmentation by distinguishing between individual objects of the same class (e.g., "Rock A" vs. "Rock B"). This is vital for manipulation tasks.

Application to Unstructured Environments:

- **Terrain Traversability:** Unlike marked roads, unstructured environments (Mars, forests) lack clear boundaries. DL segmentation learns texture-based boundaries (e.g., differentiating loose gravel from hard-packed soil).
- **Obstacle Definition:** In unstructured zones, obstacles aren't just standard shapes (cars/pedestrians). Segmentation allows the robot to identify arbitrary hazards like fallen logs or overhangs.
- **Sensor Fusion:** These networks often take RGB-D (color + depth) as input to improve segmentation accuracy in complex lighting.
- **Inference Efficiency:** For real-time robotics, lightweight variants (like MobileNet-based U-Nets) are used to run on edge hardware (e.g., NVIDIA Jetson).

Reference: Szeliski pp. 387–392, 401–405.

3. Image Morphing vs. Image Transformation

Difference:

- **Basis: Transformations** are purely geometric operations. **Morphing** is a hybrid of geometric warping and photometric interpolation.
- **Input Data:** Transformation typically acts on a **single** source image. Morphing requires **two** images (Source I_0 and Destination I_1).
- **Pixel Values:** In a transformation, pixel values (colors) are moved but generally preserved (except for resampling). In morphing, pixel values change over time (cross-dissolve).
- **Goal:** Transformation aims to change the viewpoint or align images. Morphing aims to create a smooth visual transition or animation between two different objects.

Image Transformation (Geometric):

- **Mechanism:** Maps pixel coordinates $(x, y) \rightarrow (x', y')$ using a matrix (e.g., Affine, Homography).
- **Example:** Rotating a satellite image by 90 degrees or rectifying a document scan. The content remains the same; only the geometry changes.

Image Morphing:

- **Mechanism:** Consists of two simultaneous processes:

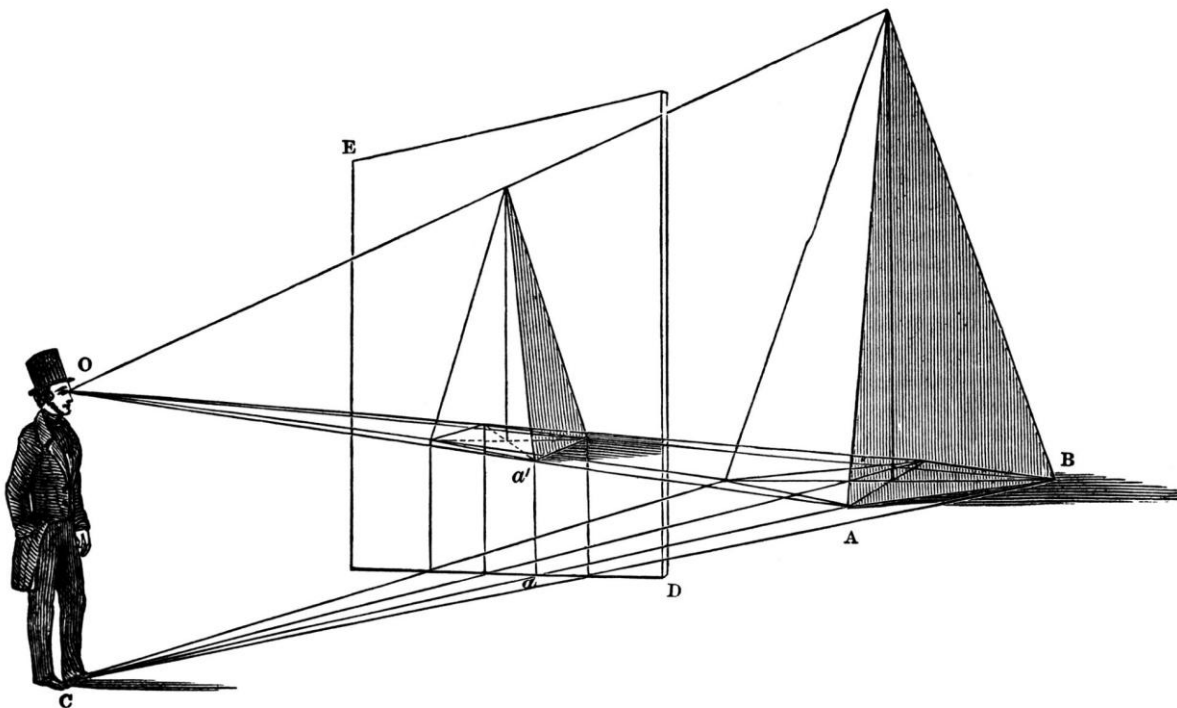
1. **Warping:** The shape of the source object is warped toward the shape of the destination object.
 2. **Cross-Dissolving:** The color of the source fades out while the destination color fades in.
- **Feature Specification:** Requires defining corresponding control points (e.g., eye to eye, nose to nose) on both images.
 - **Example:** Animating a human face turning into a tiger. The human nose widens (warp) and skin turns orange (dissolve) simultaneously.

Reference: Szeliski pp. 168–170 (*Transformations*), 177–178 (*Morphing*); Sonka p. 120.

4. Single-View Geometry and Camera Model Development

Explanation of Single-View Geometry:

- **Concept:** Recovering 3D geometric structure or camera parameters from a single 2D image by exploiting scene constraints rather than stereo parallax.
- **Vanishing Points (VPs):** Parallel lines in the 3D world converge to a single point in the 2D image plane.
- **Vanishing Lines:** The line connecting two vanishing points (e.g., the horizon line connects the VPs of horizontal ground lines).
- **Orthogonality Constraint:** If a scene contains three orthogonal directions (like the corner of a building), their three vanishing points provide strong algebraic constraints on the camera's internal geometry.



Getty Images

Importance in Camera Model Development:

- **Self-Calibration:** It allows for the estimation of the Intrinsic Matrix (K)—specifically focal length and principal point—without a laboratory calibration rig.

- **Image of the Absolute Conic (IAC):** The vanishing points of orthogonal directions lie on the IAC. Identifying them allows the robot to mathematically compute $K^{-1}K^{-T}$.
- **Metrology:** Once calibrated via single-view geometry, the robot can measure ratios of lengths (e.g., height of objects) on a ground plane.

Example:

- **Scenario:** A robot looks at a rectangular doorway.
- **Process:** The robot detects the top/bottom and side edges of the door. The intersection of top/bottom lines gives Vertical VP; sides give Horizontal VP.
- **Result:** Using these VPs, the robot calculates its focal length and can then estimate the distance to the door if the door's height is known.

Reference: Szeliski pp. 685–689, 29–33.

5. Object Detection Pipeline with Segmentation and RANSAC

Pipeline Steps:

1. **Segmentation (Region Proposal):**
 - **Purpose:** To limit the computationally expensive search to likely areas.
 - **Action:** Apply color segmentation (e.g., HSV thresholding) or background subtraction to find "blobs" or Regions of Interest (ROIs).
2. **Feature Extraction:**
 - **Purpose:** To convert the ROI pixels into robust mathematical descriptions.
 - **Action:** Detect keypoints using **SIFT** (high accuracy) or **ORB** (high speed). Compute descriptors (vectors) for each keypoint that are invariant to scale and rotation.
3. **Feature Matching:**
 - **Purpose:** To find correspondence between the live robotic view and a stored object model.
 - **Action:** Perform Nearest Neighbor search (Euclidean distance). Apply **Lowe's Ratio Test** (reject match if distance to 1st neighbor is not significantly smaller than distance to 2nd neighbor) to eliminate ambiguous matches.
4. **RANSAC-based Outlier Rejection:**
 - **Purpose:** To filter out false matches that passed the ratio test but don't make geometric sense.
 - **Action:**
 - Randomly select 4 matches.
 - Compute a Homography (planar transform).
 - Check consistency of other matches.
 - Select the transformation with the most inliers.
5. **Pose Estimation:**
 - **Action:** Use the inliers to compute the object's 6D pose relative to the camera, enabling the robot to grasp it.

6. Calibration and Single-View Geometry

Concept of Camera Calibration:

- **Definition:** The process of determining the intrinsic parameters (focal length f_x, f_y , principal point c_x, c_y , skew) and lens distortion coefficients.
- **Matrix:** Defines the mapping $x = K [R|t] X$, where K is the calibration matrix.

Influence on Single-View Geometry:

- **Straightness Preservation:** Single-view geometry relies heavily on finding Vanishing Points from straight lines. Uncalibrated cameras have **Radial Distortion** (barrel/pincushion) which bends lines. Calibration allows the robot to "undistort" the image, making straight lines straight again.
- **Metric Stratification:**
 - **Uncalibrated:** The robot views the world in **Projective** space. It can check collinearity (points on a line) but cannot measure angles or distances.
 - **Calibrated:** The robot operates in **Euclidean** space. It can measure true angles (e.g., "is this corner 90 degrees?") and metric ratios.
- **Ray Tracing:** Calibration permits **Back-Projection**. A pixel (u,v) can be converted into a normalized 3D ray (X,Y,Z) . This is essential for determining the angle between two observed features.
- **Plane Orientation:** With calibration, detecting vanishing points of a ground plane allows the computation of the plane's normal vector (the horizon orientation), telling the robot if the floor is sloped.

Reference: Szeliski pp. 56–58, 685–692; Sonka p. 602.

7. Filtering and Restoration for Noisy Datasets

Design of Stepwise Approach:

1. Noise Analysis & Identification:

- Analyze pixel intensity histograms to identify the noise model.
- **Salt-and-Pepper:** Random black/white pixels (data dropouts).
- **Gaussian:** Random variation across all pixels (sensor thermal noise).

2. Step 1: Outlier Removal (Non-Linear):

- Apply a **Median Filter**. This is crucial as the first step to remove Salt-and-Pepper noise.
- *Why:* Linear filters smear these outliers; Median filtering completely removes them while preserving edges.

3. Step 2: Denoising / Smoothing (Edge-Preserving):

- Apply a **Bilateral Filter**.
- *Why:* It averages pixels based on spatial proximity *and* photometric similarity. It smooths the Gaussian noise on flat walls but refuses to blur the edges of objects, which is critical for subsequent segmentation.

4. Step 3: Deblurring (Restoration):

- If motion blur is present, estimating the Point Spread Function (PSF) is required.
- Apply **Wiener Filtering**. This performs deconvolution to reverse the blur while regulating the amplification of any remaining noise (using the signal-to-noise ratio).

5. Step 4: Contrast Enhancement:

- Apply **CLAHE** (Contrast Limited Adaptive Histogram Equalization).
- *Why*: Robotic environments often have poor lighting. This enhances local contrast in shadows without over-amplifying noise in flat regions.

Reference: Szeliski pp. 119–136; Sonka pp. 125–133, 162–165.

◆ MODULE 2

8. Visual Odometry: Semi-direct vs Direct Sparse

Semi-Direct Visual Odometry (SVO):

- **Hybrid Nature**: Combines the speed of direct methods with the robustness of feature-based mapping.
- **Tracking Step (Direct)**: Estimates frame-to-frame motion by minimizing photometric error on small 4×4 patches around detected corners. It does *not* compute descriptors.
- **Mapping Step (Indirect)**: Once the pose is estimated, it uses probabilistic depth filters to refine 3D points, similar to feature-based methods.
- **Example**: Ideal for high-speed drone navigation (e.g., quadcopters) where processing power is limited but high frame-rate tracking is needed to prevent drift.

Direct Sparse Odometry (DSO):

- **Purely Direct**: Does not use keypoints or descriptors at all.
- **Sparse Selection**: Operates on all pixels that have sufficient intensity gradients (edges, shadows, soil texture), not just corners.
- **Optimization**: Jointly optimizes camera pose, geometry (inverse depth), and intrinsic parameters (like exposure time/gamma) by minimizing photometric error across a window of keyframes.
- **Example**: Automated driving on asphalt roads. Feature detectors fail on the road surface (no corners), but DSO can track the road texture and lane markings using the gradients.

Reference: Szeliski pp. 735, 968.

9. Bundle Adjustment and Sparsity

Bundle Adjustment (BA) Strategies:

- **Goal**: Global refinement of camera poses (P) and 3D points (X) to minimize Reprojection Error.
- **Levenberg-Marquardt (LM)**: The standard non-linear least squares optimizer. It interpolates between Gradient Descent (robust start) and Gauss-Newton (fast finish).
- **Cost Function**: $E = \sum ||x_{\text{observed}} - \pi(P, X)||^2$.

Effect of Sparsity on Efficiency:

- **Matrix Structure:** The Normal Equations ($J^T J \Delta = -J^T r$) involve the Hessian matrix. This matrix is massive but **Sparse**.
- **Why Sparse?** A single 3D point is seen by only 2-10 cameras, not all of them. A camera sees only a fraction of total scene points. There is no direct correlation between Point A and Point B.
- **Schur Complement (Marginalization):**
 - We can partition the matrix into Camera blocks and Point blocks.
 - Because point blocks are diagonal (points don't interact), we can mathematically invert them easily and subtract them from the camera equations.
- **Result:** This reduces the problem from solving a system of size $(N_{\text{cams}} + N_{\text{points}})$ to just N_{cams} .
- **Impact:** Enables "Large-Scale" SfM. Without exploiting sparsity, reconstructing a city with 10,000 points would be computationally impossible. With sparsity, it becomes efficient.

Reference: Szeliski pp. 717–722.

10. Re-localization and Large-Scale Map Optimization

Challenges in Re-localization:

- **Kidnapped Robot Problem:** The robot must determine its location without any prior pose guess (global search).
- **Perceptual Aliasing:** In large environments (e.g., office buildings), many corridors look identical. The robot may match a visual feature to the wrong location in the map.
- **Viewpoint Invariance:** The robot might revisit a room from a different angle or distance than when it first mapped it. Standard template matching fails here; invariant features (BoW) are needed.

Challenges in Large-Scale Map Optimization:

- **Drift Accumulation:** Small errors in odometry accumulate over long paths, causing the map to "break" (corridors don't connect).
- **Computational Complexity:** Bundle Adjustment scales cubically. Optimizing a map with millions of points is infeasible in real-time.
- **Loop Closure:** The system must detect when it returns to a known start point. This is difficult if lighting has changed or dynamic objects (people) are present.
- **Solution (Pose Graph Optimization):** To handle scale, full BA is abandoned. Instead, the system freezes the 3D points and optimizes a "Pose Graph"—a simplified network of camera nodes and relative constraints.

Reference: Szeliski pp. 698, 734–736.

11. SLAM: Initialization, Tracking, and Mapping

1. Initialization:

- **Task:** Bootstrap the system. Establish the first reference frame and scale.
- **Method:** Requires two frames with sufficient translation (parallax). The system computes the Essential Matrix, decomposes it to R and t , and triangulates initial landmarks.

- **Example:** A cleaning robot moves 20cm forward. It uses the shift in floor texture to define its initial "Map Unit."

2. Tracking (Front-End):

- **Task:** Locate the robot in the current map at frame rate.
- **Method:**
 - Predict pose (using motion model).
 - Project local map points into the image.
 - Search for matches.
 - Minimize reprojection error (Motion-only Bundle Adjustment) to refine pose.
- **Challenge:** Fast motion causing blur or loss of track.

3. Mapping (Back-End):

- **Task:** Extend the map with new information.
- **Method:**
 - **Keyframe Decision:** If the robot has moved enough, create a new Keyframe.
 - **Triangulation:** Search for unmatched features in neighbor keyframes to create new 3D points.
 - **Local BA:** Optimize the last N keyframes and their points to ensure consistency.
 - **Culling:** Remove redundant points or keyframes to keep memory usage bounded.

Reference: Szeliski pp. 734–739.

12. Sensor Characteristics for SLAM

1. Monocular Sensors:

- **Characteristics:** Single camera, passive light.
- **Pros:** Low cost, low power, compact, infinite range (can see distant mountains).
- **Cons:** **Scale Ambiguity** (cannot tell absolute size/distance), initialization requires movement, fails in low texture/darkness.
- **Suitability:** Good for drones/phones where weight is critical, provided IMU fusion is used to solve scale.

2. Stereo Sensors:

- **Characteristics:** Two cameras, fixed baseline.
- **Pros:** Solves scale ambiguity (depth is absolute), works outdoors, passive (no interference from sunlight).
- **Cons:** Computational cost (matching), depth range limited by baseline (blind at long distance), calibration sensitive.
- **Suitability:** The standard for outdoor robotics (autonomous cars, Mars rovers).

3. RGB-D Sensors (Structured Light/ToF):

- **Characteristics:** Infrared projector + Camera.
- **Pros:** Provides dense, accurate depth instantly even on textureless white walls.

- **Cons: Fails outdoors** (sunlight washes out IR), high power consumption, limited range (< 10m).
- **Suitability:** The gold standard for indoor service robots (warehouse bots, domestic vacuums).

Reference: Szeliski pp. 627–630, 753–756.

13. Direct vs Indirect SLAM; LSD-SLAM and ORB-SLAM

Differentiation:

- **Indirect (Feature-Based):**
 - Abstracts the image into sparse geometric features (corners/blobs).
 - Minimizes **Geometric Reprojection Error** (distance in pixels).
 - Robust to lighting and wide baselines but sparse output.
- **Direct:**
 - Uses raw pixel intensities directly.
 - Minimizes **Photometric Error** (intensity difference).
 - Produces semi-dense maps and works in low-texture but sensitive to lighting.

LSD-SLAM (Large-Scale Direct SLAM):

- **Type:** Direct, Monocular.
- **Strengths:** Reconstructs not just corners but all edges and high-gradient areas, providing a map that is visually interpretable for humans (walls, furniture outlines).
- **Suitability:** Good for AR and environments with sparse features but good gradients (corridors).

ORB-SLAM (Feature-Based):

- **Type:** Indirect, Monocular/Stereo/RGB-D.
- **Strengths:** Extremely robust. Uses ORB features which are fast and invariant to view/light. Includes "Loop Closure" and "Relocalization" capabilities that LSD-SLAM often lacks in robustness.
- **Suitability:** The default choice for general-purpose robotic navigation due to its reliability.

Reference: Szeliski pp. 735, 967–968.

14. Feature-based Alignment and Pose Estimation

Categories of Alignment:

1. **2D-2D Alignment (Image Stitching / Initialization):**
 - **Input:** Correspondences between two 2D images.
 - **Technique:** Estimation of **Homography** (for planar scenes/rotation only) or **Fundamental Matrix** (for general motion).
 - **Example:** A robot stitching images to create a panoramic map, or initializing VO from the first two frames.
2. **3D-2D Alignment (Pose Tracking / PnP):**
 - **Input:** Known 3D Map points and current 2D image features.

- **Technique: Perspective-n-Point (PnP)** algorithms (e.g., P3P, EPnP). Minimizes the reprojection error of 3D points onto the 2D plane.
- **Example:** A robot entering a known room. It matches the "door frame" (3D map) to its camera view (2D) to find exactly where it is standing.

3. 3D-3D Alignment (Map Merging / Registration):

- **Input:** Two clouds of 3D points (e.g., from Stereo or RGB-D).
- **Technique: Iterative Closest Point (ICP).** Aligns shape A to shape B by iteratively minimizing the distance between nearest neighbors.
- **Example:** Loop closure. When the robot returns to start, it aligns its current local 3D map with the initial 3D map to correct drift.

Reference: Szeliski pp. 503–505, 693–695, 715.

15. Geometric Error Formulation in Visual SLAM

Role of Error Formulation:

- The error function defines the "objective" the optimization engine (Bundle Adjustment) tries to minimize. It dictates the accuracy and robustness of the SLAM system.

Indirect (Geometric) Error:

- **Formulation:**
$$e = || u_{\text{measured}} - \pi(P \cdot X_{\text{world}}) ||^2$$
- **Explanation:** Measures the Euclidean distance (in pixels) between where a feature detector *found* a point and where the math says it *should* be.
- **Role:** Decouples lighting from geometry. Allows for wide-baseline matching because descriptors (SIFT/ORB) handle the appearance matching, leaving the optimizer to handle only geometry.

Direct (Photometric) Error:

- **Formulation:**
$$e = || I_{\text{ref}}(u) - I_{\text{curr}}(\pi(u)) ||^2$$
- **Explanation:** Measures the difference in brightness/color. It assumes a point in the world looks the same from slightly different angles (Brightness Constancy).
- **Role:** Allows sub-pixel accuracy (geometry is continuous, not discrete pixels). Exploits info from edges and smooth gradients that feature detectors ignore.
- **Constraint:** Requires high frame rates (small motion) so that the appearance doesn't change too much between frames.

Reference: Szeliski pp. 528, 718 (Geometric), 558 (Photometric).

◆ MODULE 3

16. Active Learning in Robotic Vision Recognition

Analysis:

- **Definition:** A training strategy where the robot algorithm allows the learner to choose the data from which it learns.

- **Key Driver:** Labeling data is expensive; robot video feeds are infinite. Active learning filters the "boring" data (99% of frames) to find the "interesting" data (1% of frames).
- **Uncertainty Sampling:** The robot evaluates its own confidence. If it detects an object with 50% probability "Car" and 50% "Truck," it flags this frame for human review.
- **Diversity Sampling:** The robot ensures the training set covers all visual domains (e.g., ensuring it doesn't just learn "Daytime" driving but specifically requests "Night" or "Rain" images if the dataset lacks them).
- **Human-in-the-Loop:** Facilitates continuous learning. The robot asks for help only when needed, and the human feedback immediately updates the model.
- **Real-Time Application (Novelty Detection):** A planetary rover detecting a rock with a spectral signature unlike anything in its database. It prioritizes this image for transmission, "actively" deciding it is scientifically important.
- **Real-Time Application (Domain Adaptation):** A warehouse robot moved to a new facility with different lighting. It uses active learning to request labels for the new environment's specific obstacles, adapting in hours rather than weeks.

Reference: Szeliski pp. 266–268, 406.

17. Sequence Modelling for Vision Interpretation

Concept:

- Robotic vision is inherently temporal. Single-frame analysis ignores motion, causality, and persistence. Sequence modelling integrates information over time t .

Deep Learning Mechanisms:

- **RNNs / LSTMs:** Networks with "memory" (hidden states). They process video frame-by-frame, updating their belief about the world. Good for tracking and low-latency predictions.
- **Transformers (Attention):** Process chunks of video simultaneously. "Self-attention" allows the model to relate an event at $t=100$ to an event at $t=1$, capturing long-term dependencies.

Applications:

- **Action Recognition:** Distinguishing "Opening a door" from "Closing a door." Both single frames look identical; only the *sequence* reveals the action.
- **Trajectory Prediction:** A robot observing a pedestrian. The sequence model analyzes the past 10 steps to predict the next 10, enabling the robot to plan a collision-free path.
- **Visual Odometry (DeepVO):** End-to-end Deep Learning VO uses RNNs to learn the physics of motion directly from video clips, estimating pose without explicit feature matching.
- **Object Permanence:** If a ball rolls behind a sofa, a standard CNN forgets it. An RNN remembers the "ball state" even when occluded, predicting where it will re-emerge.

Reference: Szeliski pp. 321, 396–400.

18. Uncertainty Estimation in Perception and Decision-Making

Why Crucial?

- **Safety:** Standard Deep Nets are often "confidently wrong" on out-of-distribution data. A robot must know *when* it doesn't know to fail safely (e.g., stop the car).
- **Sensor Fusion:** Uncertainty provides the "weights" for fusion. (e.g., "Vision is 20% confident, Lidar is 90% confident" $\rightarrow$ Trust Lidar).

Types:

- **Aleatoric:** Uncertainty in the data (noise, blur, darkness).
- **Epistemic:** Uncertainty in the model (never seen this object before).

Examples of Impact on Decision Making:

- **Autonomous Driving:** The perception system detects an object ahead.
 - *High Certainty:* "It is a plastic bag." $\rightarrow$ Decision: Drive over it.
 - *High Uncertainty:* "It might be a rock or a bag." $\rightarrow$ Decision: Swerve/Stop.
- **Grasping:** A robot arm picks up a glass.
 - *High Certainty:* Fast, ballistic movement.
 - *High Uncertainty (Slippage risk):* Slow approach, activate tactile feedback loop, apply higher grip force.
- **Exploration:** A mapping robot sees a dark corridor (High Epistemic Uncertainty).
 - *Decision:* Instead of avoiding it, the uncertainty *attracts* the robot to explore that area to reduce map entropy.

Reference: Szeliski pp. 723, 952 (Appendix B.6).